8 Clustering

Clustering groups observations that "look similar" without using any external labels. In contrast to classification, where each sample is mapped to a known class, clustering is unsupervised—the algorithm must discover structure on its own. Figure 8.9 illustrates this distinction: the left panel shows a labeled decision surface learned from annotated data (classification), while the right panel shows the same points partitioned solely by proximity, with no labels supplied (clustering). Clustering is valuable when you

1. have no ground-truth labels
2. suspect there are hidden sub-populations
3. want a quick exploratory summary before choosing a supervised model.

Review 8.1 Wine Dataset
The wine dataset contains 178 samples of Italian red wine from three cultivars grown in the same region. Each sample is represented by 13 continuous measurements, such as alcohol content, flavonoid concentration, magnesium level, and color intensity. These measurements capture chemical and physical differences among the wines. Since the three cultivars form natural but partially overlapping groups, the wine dataset is commonly used as a benchmark for clustering and other multivariate analysis methods.

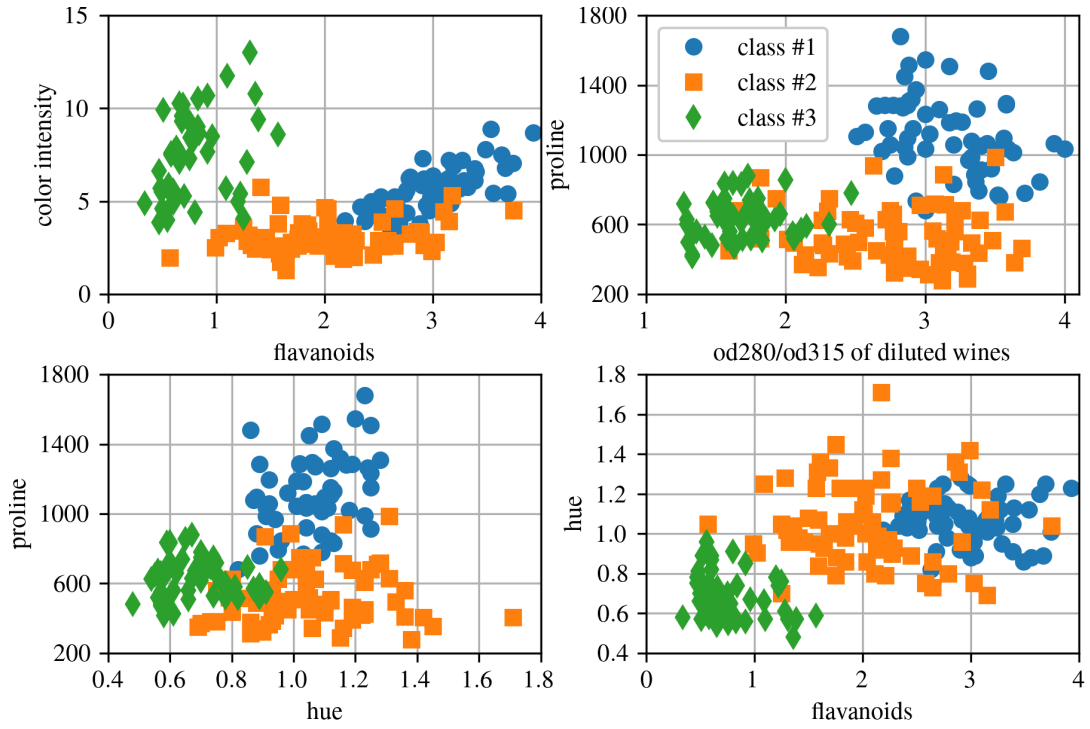

Figure 8.8: Two feature-feature plots developed for the wine data set.

A clustering algorithm receives only the feature matrix

$$X = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]^T, \tag{8.8}$$

where each observation $\mathbf{x}_i \in \mathbb{R}^d$ is described by d measured or engineered features. Unlike supervised learning, there is no target vector $\mathbf{y}$ that tells the algorithm the correct answer. The goal is therefore not to reproduce known labels, but to discover structure that may be useful for interpretation, visualization, preprocessing, or later supervised modeling.

The word "similar" is important. Most clustering algorithms depend on a distance or similarity measure, such as Euclidean distance,

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2. \tag{8.9}$$

If one feature has a much larger numerical range than the others, it can dominate the distance calculation. For this reason, clustering is usually applied after feature scaling, particularly when the features have different units. In scikit-learn this is commonly handled by placing a transformer such as `StandardScaler` before the clustering estimator in a pipeline.

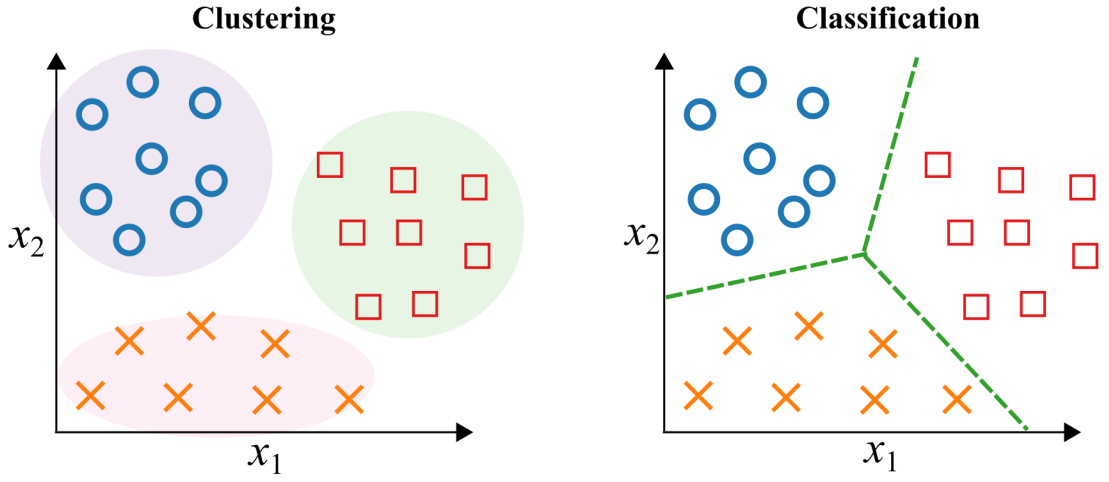

Figure 8.9: A comparison between clustering on the left and classification on the right.

Figure 8.10 summarizes several common clustering ideas. The methods in this chapter differ mainly in how they define a cluster. K-means represents each cluster by a centroid. Hierarchical clustering builds a tree of nested groups. DBSCAN and HDBSCAN define clusters as dense regions separated by sparse regions. Gaussian mixture models treat the data as being generated by a weighted combination of probability distributions. These different assumptions lead to different strengths and weaknesses.

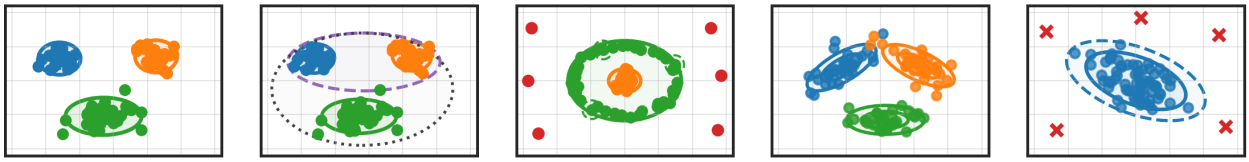

Figure 8.10: Examples of several clustering methods.

8.1 K-Means

K-means is one of the most widely used clustering algorithms. It partitions a data set into a chosen number of clusters, k, by assigning each observation to the closest cluster center. Each cluster center is called a centroid because it is computed as the mean location of all observations assigned to that cluster. In scikit-learn, this model is implemented by `sklearn.cluster.KMeans`.

K-means is simple, fast, and easy to interpret. Its main assumption is that each cluster can be represented well by a single center and that points should be grouped by Euclidean distance from those centers. This works well for compact, roughly circular or spherical clusters. It works less well when clusters are curved, elongated, strongly overlapping, or have very different densities.

Throughout this section, the examples use the wine data set represented by two standardized features:

$$x_1 = \texttt{color\_intensity}, \qquad x_2 = \texttt{proline}.$$

The original wine data set includes three known wine classes. These class labels are useful for visualization, but they are not used by K-means. K-means only sees the feature values and tries to find groups based on distance.

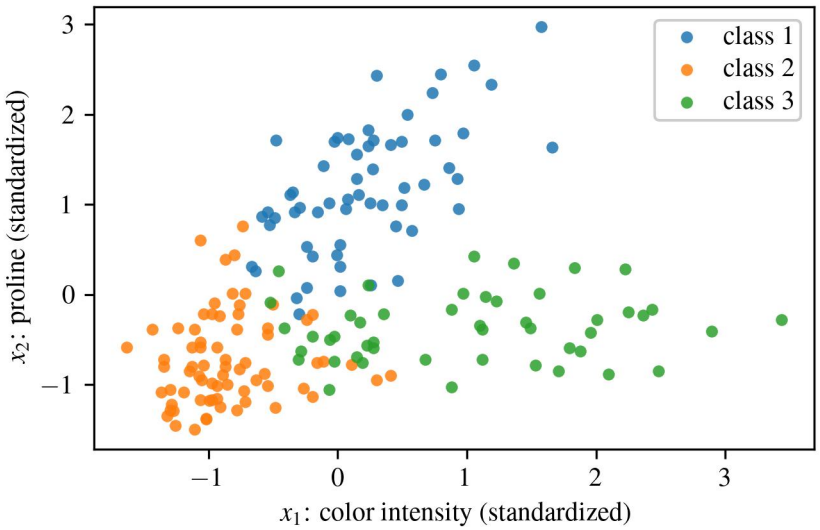

Figure 8.11: Wine data represented using two standardized features: color intensity and proline. Points are colored by the known wine class labels. These labels are shown only for interpretation and are not provided to K-means during clustering.

8.1.1 Objective Function

Problem setup Given observations

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d,$$

K-means seeks k centroids

$$\mu_1, \mu_2, \ldots, \mu_k$$

and cluster assignments

$$c_i \in \{1, 2, \ldots, k\}$$

that minimize the within-cluster sum of squared distances:

$$J = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \mu_{c_i} \right\|^2. \tag{8.10}$$

The objective J is often called the inertia or within-cluster sum of squares. A small value of J means that points are close to the centroids of their assigned clusters. Therefore, minimizing J encourages compact clusters.

The cluster label c_i is just an index. It does not necessarily correspond to a physical class label. For example, a K-means cluster labeled "cluster 1" does not automatically mean "wine class 1." The cluster numbers are arbitrary and can change depending on the initialization.

8.1.2 Assignment and Update Steps

The most common optimization procedure for K-means is Lloyd's algorithm. It alternates between two simple steps: assign points to the nearest centroid, then update each centroid to the mean of the points assigned to it.

1. **Initialize the centroids.** Choose k starting centroids, often randomly or using the k-means++ initializer.

2. **Assignment step.** Assign each point to its closest centroid:

$$c_i = \arg \min_{j \in \{1,\ldots,k\}} \left\| \mathbf{x}_i - \mu_j \right\|^2. \tag{8.11}$$

3. **Update step.** Recompute each centroid as the average of the points assigned to it:

$$\mu_j = \frac{1}{|C_j|} \sum_{i : c_i = j} \mathbf{x}_i, \tag{8.12}$$

   where C_j is the set of points assigned to cluster j.

4. **Repeat.** Continue alternating between the assignment and update steps until the assignments stop changing or the decrease in J becomes very small.

The assignment step divides the feature space into regions. Every point in one region is closer to one centroid than to the others. These regions are called Voronoi cells or decision regions. Figure 8.12 shows the final decision regions for K-means with $k = 3$ on the two-dimensional wine data.

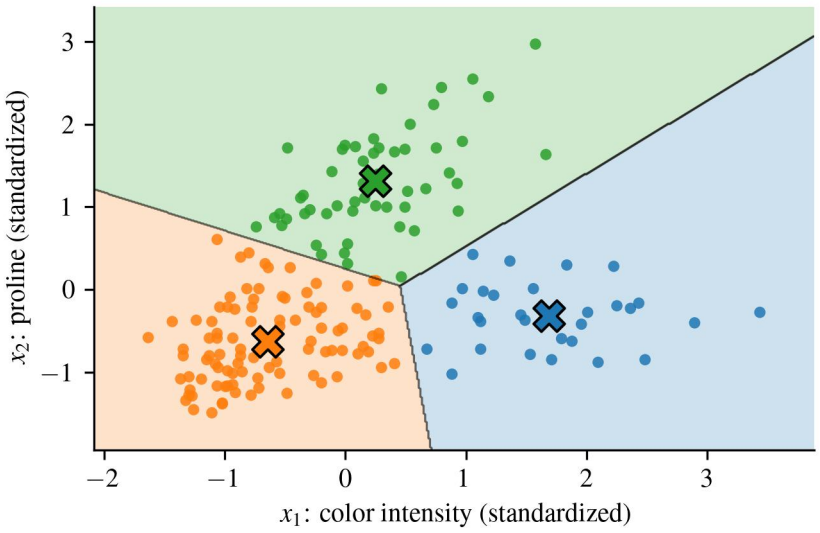

Figure 8.12: Decision regions produced by K-means after convergence. Each background color shows the region closest to one centroid, and each point is colored by its assigned K-means cluster. The large markers show the final centroid locations.

Figure 8.13 shows how the centroids move during the Lloyd iterations. The initial centroids are selected first, then the algorithm repeatedly assigns points and updates the centroid locations. After several iterations, the centroids settle into stable positions.

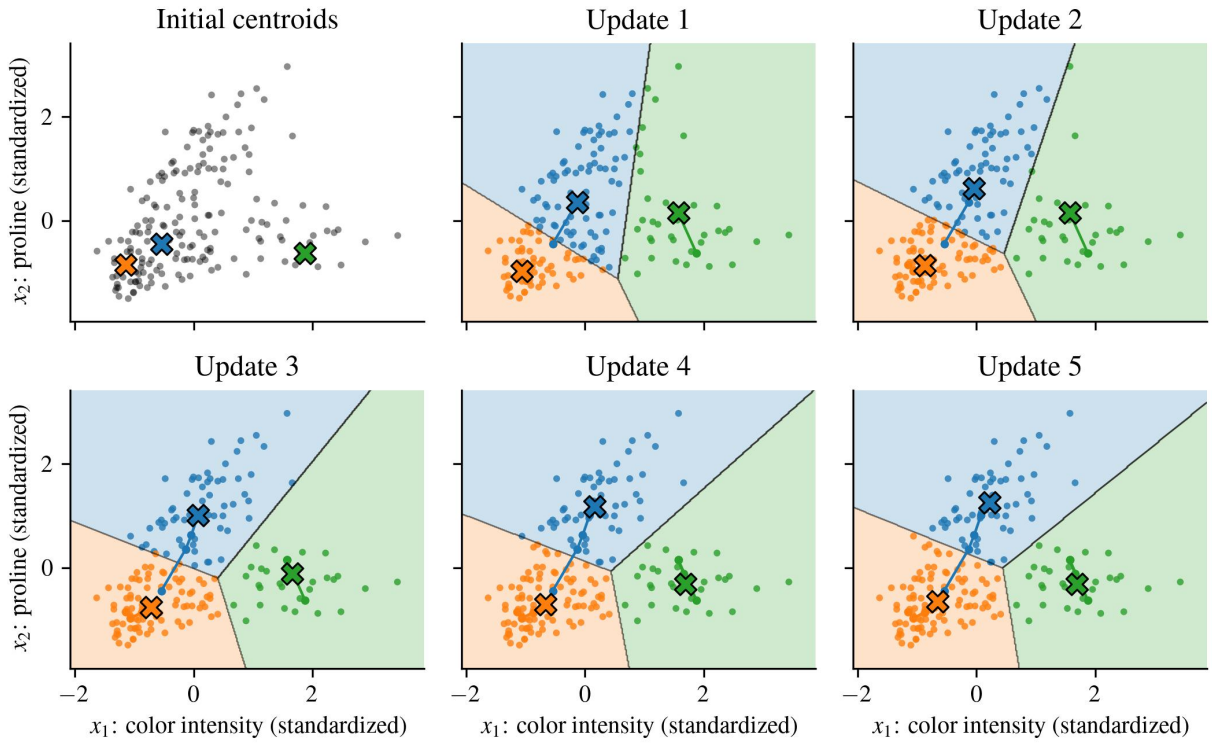

Figure 8.13: Centroid movement during K-means. The first panel shows the initial centroids. Later panels show how the centroids move as the assignment and update steps are repeated.

Each assignment step and update step does not increase the objective function in Equation 8.10. Therefore, the algorithm eventually converges. However, the objective is not convex, so K-means is not guaranteed to find the global minimum. It can converge to different solutions depending on the initial centroid locations.

Figure 8.14 shows this effect. Both panels use $k = 3$, but different random starts lead to different final clusterings. The lower-inertia solution has a smaller value of J, meaning the points are closer to their assigned centroids on average. The higher-inertia solution is a poorer local minimum.

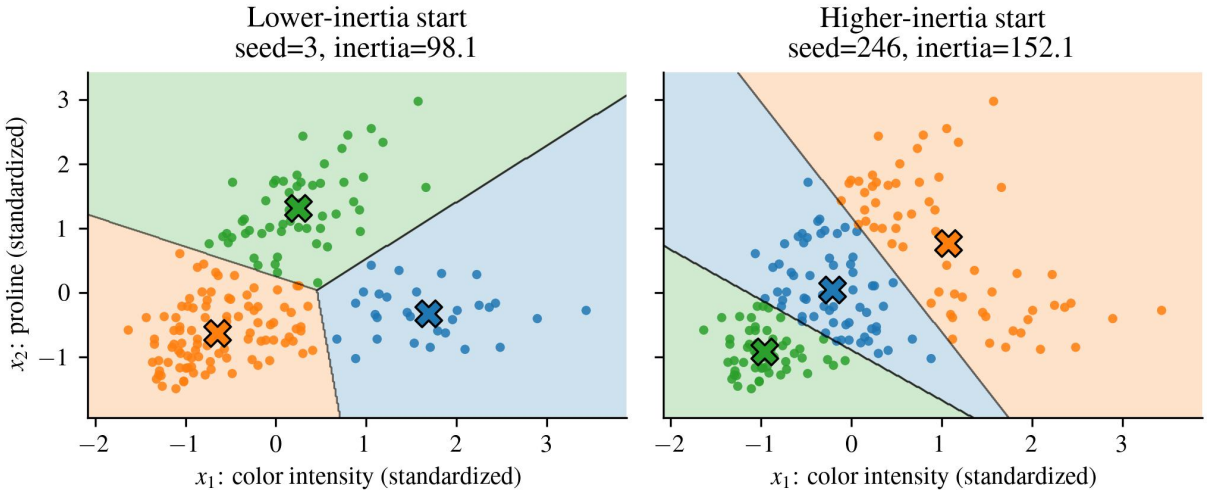

Figure 8.14: Different random initializations can lead to different K-means solutions. The lower-inertia solution is more compact, while the higher-inertia solution is a poorer local minimum.

A common practical fix is to run K-means multiple times with different initial centroids and keep the solution with the smallest inertia. Another common approach is to use the k-means++ initializer, which chooses more spread-out starting centroids and often improves the final result.

8.1.3 Choosing the Number of Clusters

The number of clusters, k, must be chosen before fitting K-means. If k is too small, distinct groups may be merged together. If k is too large, one meaningful group may be split into several artificial clusters.

One common diagnostic is the elbow rule. K-means is fit for several values of k, and the final inertia is plotted against k. Adding more clusters almost always decreases inertia, so the goal is not simply to find the smallest value. Instead, the elbow rule looks for the value of k where the improvement begins to level off.

The decrease in inertia from adding one more cluster can be written as

$$\Delta J(k) = J(k-1) - J(k). \tag{8.13}$$

A large value of $\Delta J(k)$ means that adding another cluster helped a lot. A small value means that adding another cluster gave only a small improvement.

Another useful diagnostic is the silhouette score. For point i, let a_i be the average distance from point i to other points in its assigned cluster. Let b_i be the smallest average distance from point i to the points in any other cluster. The silhouette value is

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}. \tag{8.14}$$

A silhouette value near 1 means that the point is much closer to its own cluster than to neighboring clusters. A value near 0 means that the point lies near a cluster boundary. A negative value

suggests that the point may be assigned to the wrong cluster. The mean silhouette score over all observations can be used to compare different choices of k.

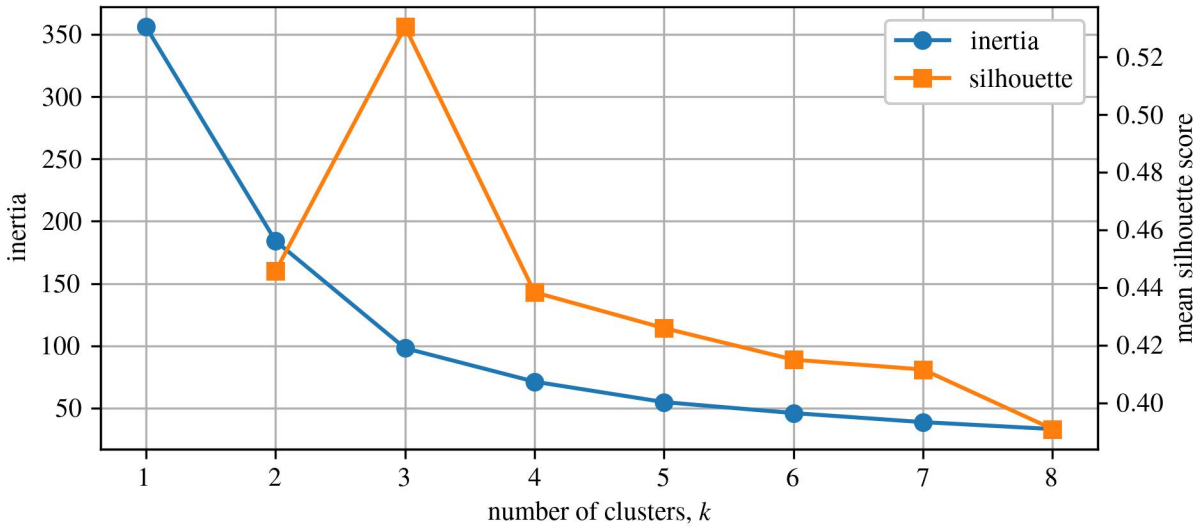

Figure 8.15: Cluster diagnostics for the wine data. Inertia decreases as k increases, while the mean silhouette score can help identify values of k that produce well-separated clusters.

For the two-dimensional wine example, the diagnostics in Figure 8.15 support $k = 3$. This is consistent with the three known wine classes shown in Figure 8.11. In real unsupervised problems, however, the true number of groups is usually not known. The final choice of k should therefore combine numerical diagnostics, visualization, and domain knowledge.

8.1.4 Practical Notes

K-means is usually efficient. For n observations, k clusters, and d features, one Lloyd iteration costs approximately

$$O(nkd). \tag{8.15}$$

In practice, the algorithm often converges in a small number of iterations.

Feature scaling is important. Since K-means uses Euclidean distance, a feature with a large numerical range can dominate the clustering. Standardizing continuous features before fitting K-means is often a good default choice.

Finally, K-means should be interpreted as a distance-based grouping method, not as a classifier. In the wine example, the known class labels help us judge whether the clusters make sense, but the algorithm itself does not use those labels. It simply groups points that are close together in the selected feature space.

8.1.5 K-Means for Image Segmentation

K-means is not limited to scatter plots or low-dimensional synthetic data. One common application is image segmentation. In this setting, each pixel is treated as an observation and the pixel values

are treated as features. For a color image, each pixel may be represented by its red, green, and blue intensities,

$$\mathbf{x}_i = \begin{bmatrix} R_i & G_i & B_i \end{bmatrix}^T. \tag{8.16}$$

K-means can then group pixels with similar colors. Replacing each pixel by its centroid color produces a compressed or segmented version of the image. This is a useful example because it shows that clustering can be used as a preprocessing step rather than as a final model.

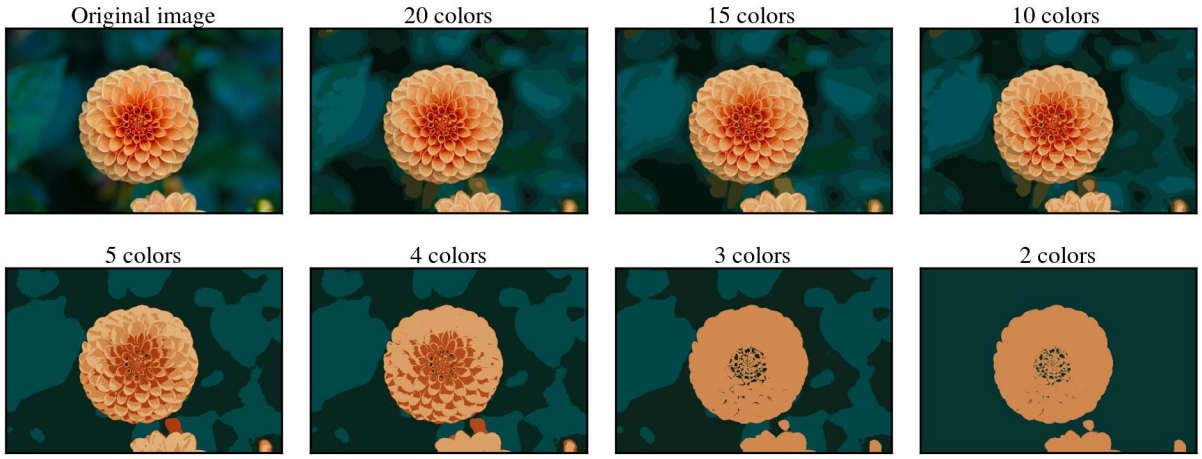

Figure 8.16: Image segmentation using K-means clustering, where pixels are grouped according to similarity in color space.

8.1.6 Limitations of K-Means

K-means works best when clusters are compact, approximately spherical, and similar in size. It can perform poorly when clusters have irregular shapes, very different densities, or strong overlap. It is also sensitive to outliers because the centroid is a mean, and means can be pulled away from the center of a group by extreme observations.

These limitations are not necessarily failures of the algorithm. They are consequences of the model assumptions. K-means assumes that squared distance to a centroid is an appropriate measure of cluster membership. When that assumption matches the data, K-means can work very well. When the data contain curved clusters, nested clusters, or clusters with different densities, other methods may be more appropriate.

8.2 Hierarchical Agglomerative Clustering

Hierarchical clustering builds a nested sequence of clusters instead of returning only one fixed partition. This is useful when the number of clusters is not known in advance or when the data may contain structure at multiple levels of detail. For example, machine condition data might first divide into normal and abnormal operation, and the abnormal group might then divide into several specific fault types.

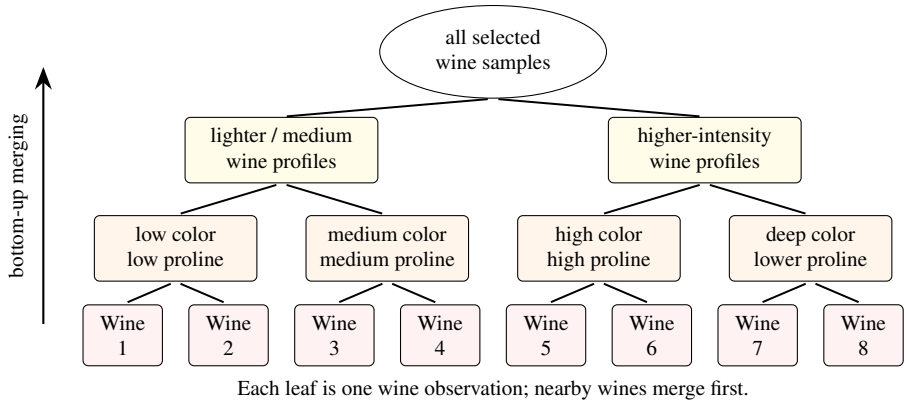

Figure 8.17: Hierarchical agglomerative clustering using a bottom-up approach on representative wine observations. Each wine starts as its own cluster, and chemically similar wines (for example, using standardized color intensity and proline) are repeatedly merged into larger groups.

Agglomerative hierarchical clustering is a bottom-up approach. It begins by treating every observation as its own cluster. The algorithm then repeatedly merges the two closest clusters until all observations belong to a single cluster. The result is a hierarchy of possible clusterings rather than a single value of k. In scikit-learn, this model is implemented by `sklearn.cluster.AgglomerativeClustering`.

The basic algorithm is:

1. Start with n clusters, one for each observation.
2. Compute the distance between every pair of clusters.
3. Merge the two closest clusters.
4. Update the distances between the new cluster and all remaining clusters.
5. Repeat until all observations have been merged into one cluster.

The key modeling decision is how to define the distance between two clusters. If A and B are two clusters, and $d(\mathbf{x}_i, \mathbf{x}_j)$ is the distance between two observations, common linkage rules include single linkage,

$$D_{\text{single}}(A,B) = \min_{\mathbf{x}_i \in A, \mathbf{x}_j \in B} d(\mathbf{x}_i, \mathbf{x}_j), \tag{8.17}$$

complete linkage,

$$D_{\text{complete}}(A,B) = \max_{\mathbf{x}_i \in A, \mathbf{x}_j \in B} d(\mathbf{x}_i, \mathbf{x}_j), \tag{8.18}$$

and average linkage,

$$D_{\text{average}}(A,B) = \frac{1}{|A||B|} \sum_{\mathbf{x}_i \in A} \sum_{\mathbf{x}_j \in B} d(\mathbf{x}_i, \mathbf{x}_j). \tag{8.19}$$

Single linkage compares the closest points in two clusters. This can capture long, chain-like clusters, but it can also connect groups through a small number of bridging points. Complete linkage compares the farthest points in two clusters, which tends to produce compact groups. Average linkage is a compromise because it uses all pairwise distances between the two clusters.

Ward linkage uses a different idea. Instead of directly measuring pairwise distances between clusters, Ward linkage merges the two clusters that lead to the smallest increase in within-cluster variation. If C is a cluster with centroid μ_C, its within-cluster sum of squares is

$$W(C) = \sum_{\mathbf{x}_i \in C} \|\mathbf{x}_i - \mu_C\|^2 . \tag{8.20}$$

When clusters A and B are merged, Ward linkage considers the increase

$$\Delta W = W(A \cup B) - W(A) - W(B). \tag{8.21}$$

This makes Ward linkage conceptually similar to K-means because both methods favor compact clusters with small within-cluster variation.

8.2.1 Dendrograms

A dendrogram is a tree diagram that shows the order in which clusters are merged. At the bottom of the dendrogram, each observation begins as its own cluster. As the height increases, clusters merge together. The height at which two branches merge represents the dissimilarity between those clusters. Cutting the dendrogram at a chosen height produces a particular clustering solution.

The advantage of a dendrogram is that it lets the user inspect the clustering structure before committing to a specific number of clusters. Large vertical gaps in the dendrogram suggest that clusters were merged only after a large increase in dissimilarity. Cutting through one of these gaps often gives a reasonable partition of the data.

Hierarchical clustering is especially useful for exploratory analysis and visualization. However, it can be computationally expensive for large data sets because many pairwise distances must be stored or updated. As with K-means, feature scaling is usually important because the distance metric determines which observations appear close together.

8.3 Density-Based Methods (DBSCAN and HDBSCAN)

Density-based clustering methods define clusters as dense regions of the feature space separated by sparse regions. This is a different assumption from K-means. K-means represents each cluster by a centroid and assigns each observation to the nearest centroid. DBSCAN, by contrast, does not begin with a centroid. Instead, it asks whether a point has enough nearby neighbors to be considered part of a dense region.

Figure 8.18 illustrates why this difference matters. The top row uses the `make_moons` dataset, and the bottom row uses the `make_circles` dataset. Both examples contain non-spherical clusters. K-means struggles because the clusters are curved, so assigning points to the nearest center cuts across the natural geometry of the data. DBSCAN performs better because it follows connected regions of high point density.

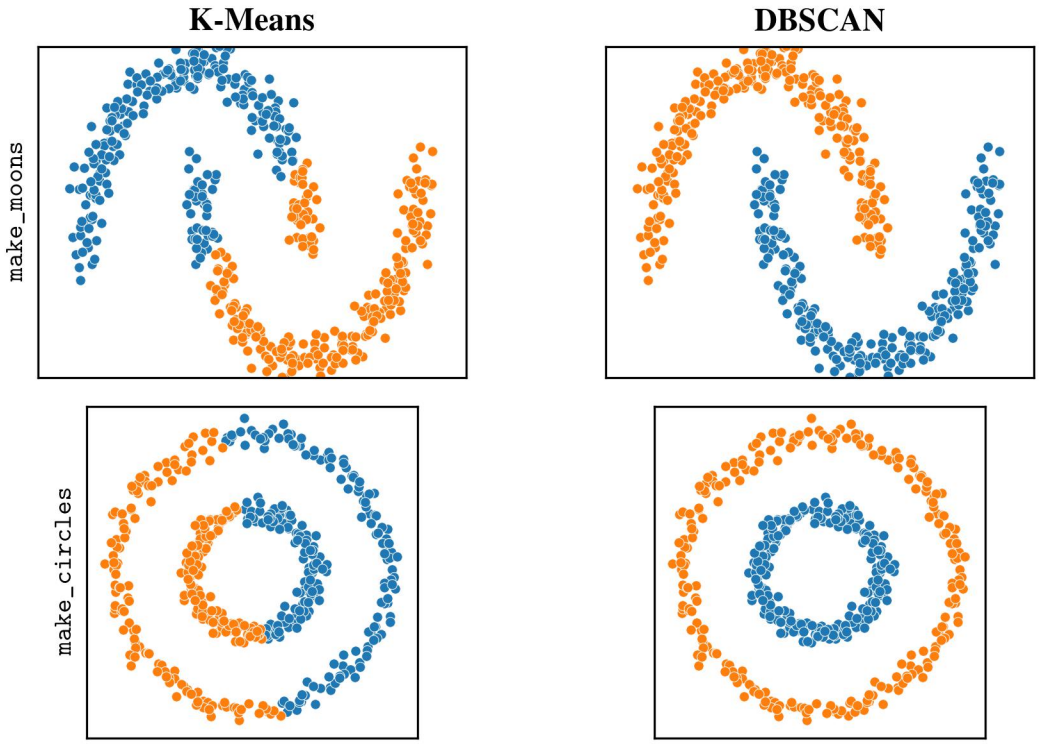

Figure 8.18: Comparison of K-means and DBSCAN on non-spherical data. K-means assigns observations to the nearest centroid, while DBSCAN groups connected dense regions.

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise. In scikit-learn, it is implemented by `sklearn.cluster.DBSCAN`. The method has two main parameters:

- ε, which defines the radius of a local neighborhood.
- minPts, called `min_samples` in scikit-learn, which defines how many points must be nearby for a region to be considered dense.

For a point $\mathbf{x}_i$, the ε-neighborhood is the set of all points within distance ε:

$$N_\varepsilon(\mathbf{x}_i) = \left\{ \mathbf{x}_j \in X : d(\mathbf{x}_i, \mathbf{x}_j) \leq \varepsilon \right\}. \tag{8.22}$$

Here $d(\mathbf{x}_i, \mathbf{x}_j)$ is usually Euclidean distance. A point is called a core point if its neighborhood contains at least minPts points:

$$|N_\varepsilon(\mathbf{x}_i)| \geq \text{minPts}. \tag{8.23}$$

This definition is the main idea behind DBSCAN. A core point sits in a dense part of the data. A border point is not dense enough to be a core point, but it lies close to a core point. A noise point is neither a core point nor close enough to a core point to be attached to a cluster.

DBSCAN builds clusters by connecting core points. If two core points are within distance ε, they are part of the same dense region. If a chain of core points connects one part of the data to another, all of those points are assigned to the same cluster. Border points are then attached to nearby clusters, while noise points remain unassigned.

The choice of ε is important. Figure 8.19 shows what happens when ε is too small. The neighborhoods are so small that most points do not have enough neighbors to become core points. As a result, DBSCAN labels most of the data as noise. This does not mean the data have no structure; it means the density scale chosen by ε is too small for this dataset.

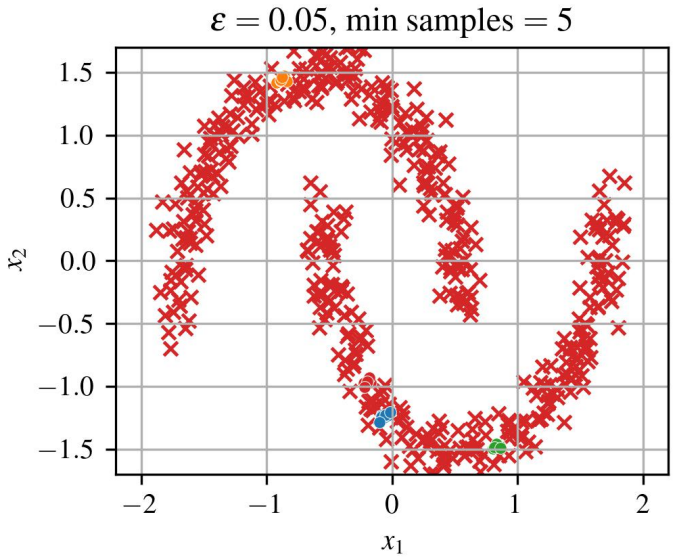

Figure 8.19: DBSCAN with ε too small. Most observations are labeled as noise because each point has too few neighbors inside its local neighborhood.

Figure 8.20 shows a more useful value of ε. The transparent circles represent local ε-neighborhoods around core points. These neighborhoods overlap along each moon-shaped region, allowing DBSCAN to connect the points into two clusters. A few isolated observations remain labeled as noise because they do not belong to any dense region.

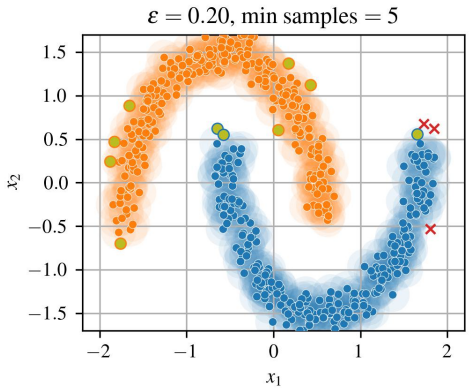

Figure 8.20: DBSCAN with a larger ε, where overlapping neighborhoods connect core points into two curved clusters. Points that do not belong to a dense region are labeled as noise.

The three figures together show the basic DBSCAN story. K-means is a centroid-based method,

so it works best when clusters can be described by their centers. DBSCAN is a density-based method, so it works best when clusters can be described as connected dense regions. This makes DBSCAN useful for curved clusters, irregular shapes, and datasets that contain outliers.

However, DBSCAN is sensitive to the value of ε. If ε is too small, many points are labeled as noise. If ε is too large, separate clusters may be merged together. The parameter minPts also matters. Larger values of minPts require denser neighborhoods and can make the method more conservative. Smaller values allow clusters to form more easily but may also make the model more sensitive to noise.

A common way to choose ε is to use a nearest-neighbor distance plot. For each observation, compute the distance to its k-th nearest neighbor, where k is usually chosen to match minPts. These distances are sorted from smallest to largest. A sharp bend in the curve suggests a transition between dense regions and sparse regions, and this bend can be used as a starting value for ε.

HDBSCAN extends DBSCAN by reducing the need to choose one fixed value of ε. Instead of clustering the data at a single neighborhood radius, HDBSCAN builds a hierarchy of density-based clusterings over many density levels. Clusters that remain stable over a range of density levels are kept, while unstable clusters are discarded. This makes HDBSCAN useful when different clusters have different densities.

One way to understand HDBSCAN is through the core distance. Let $d_c(\mathbf{x}_i)$ be the distance from $\mathbf{x}_i$ to its minPts-th nearest neighbor. Points in dense regions have small core distances, while points in sparse regions have large core distances. HDBSCAN uses this idea to make sparse points appear farther apart and dense points appear more strongly connected.

A common distance used in HDBSCAN is the mutual reachability distance:

$$d_m(\mathbf{x}_i, \mathbf{x}_j) = \max\left\{d_c(\mathbf{x}_i), d_c(\mathbf{x}_j), d(\mathbf{x}_i, \mathbf{x}_j)\right\}. \tag{8.24}$$

This distance increases the separation between sparse points while preserving connections inside dense regions. The algorithm then builds a hierarchy from this density-adjusted distance structure.

8.3.1 When to Use Density-Based Clustering

Density-based clustering is a good choice when the clusters may have irregular shapes or when the data contain outliers. It is often useful for spatial data, sensor data, and operating-regime discovery. For example, a machine may spend most of its time in several dense regions corresponding to common operating states, while unusual transients or faults may appear as sparse points between those regions.

DBSCAN is usually easiest to explain and visualize because its two parameters have a direct interpretation: ε sets the neighborhood size, and minPts sets how dense a neighborhood must be. HDBSCAN is often more flexible in practice because it can handle clusters with different densities. In both cases, the result still depends on the feature representation and distance metric, so feature scaling should usually be applied before fitting the model.

8.4 Gaussian Mixture Models

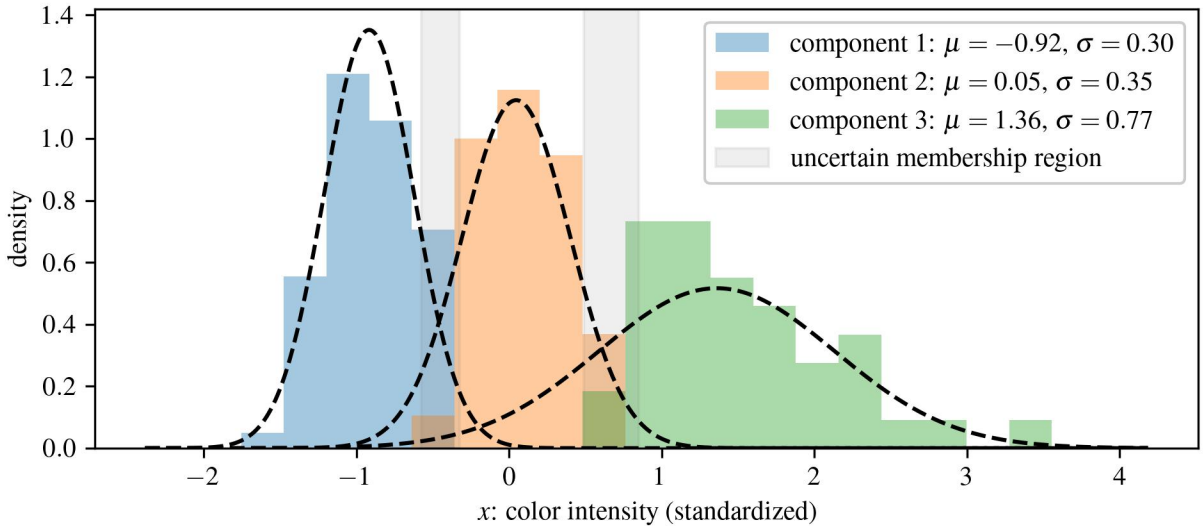

Figure 8.21: General idea of Gaussian mixture modeling applied to the wine data using standardized color intensity. The histograms show wine observations grouped by their most likely Gaussian component, the dashed curves show the fitted component densities, and the shaded regions indicate feature values where component membership is uncertain.

K-means assigns each observation to exactly one cluster. Gaussian mixture models provide a probabilistic alternative. Figure 8.21 shows the general idea using the wine data: instead of placing each wine only into a single centroid-based group, the model represents the data as a weighted combination of Gaussian components. Each component has its own mean and spread, shown by the dashed Gaussian curves in the figure.

This probabilistic view is useful when groups overlap. A wine observation near the overlap between two components may not clearly belong to only one group. Instead, the model can assign probabilities that describe how strongly the observation belongs to each component. More generally, each Gaussian component has its own mean and covariance matrix, which allows clusters to have different shapes, orientations, and sizes.

A Gaussian mixture model with k components has probability density

$$p(\mathbf{x}) = \sum_{j=1}^{k} \pi_j \mathcal{N}\left(\mathbf{x} \mid \mu_j, \Sigma_j\right), \tag{8.25}$$

where π_j is the mixing weight for component j, μ_j is the component mean, and Σ_j is the component covariance matrix. The mixing weights satisfy

$$\pi_j \geq 0, \qquad \sum_{j=1}^{k} \pi_j = 1. \tag{8.26}$$

The multivariate Gaussian density is

$$\mathcal{N}(\mathbf{x} \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\mu)^T\Sigma^{-1}(\mathbf{x}-\mu)\right]. \tag{8.27}$$

The mean μ controls the center of the Gaussian component, while the covariance matrix Σ controls its spread, shape, and orientation. This makes Gaussian mixture models more flexible than K-means. K-means effectively represents each cluster with only a center, while a Gaussian mixture model represents each component with both a center and a covariance structure.

In scikit-learn, Gaussian mixture models are implemented by `sklearn.mixture.GaussianMixture`. The fitted model can be used to predict cluster labels, estimate probabilities of component membership, sample new observations, or evaluate the likelihood of new data.

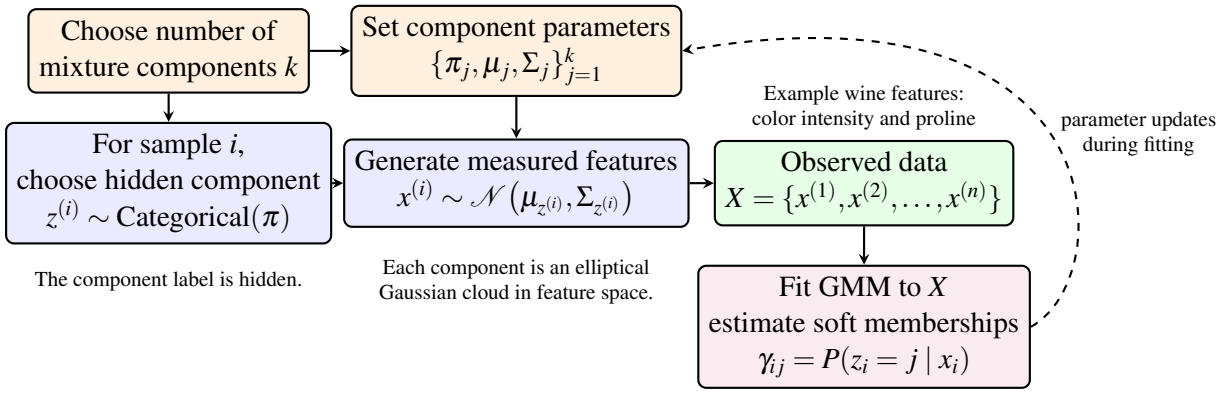

Figure 8.22: Flow-chart representation of a Gaussian mixture model for the wine data. The model assumes that each wine is associated with an unobserved mixture component, and then its measured feature vector is generated from the Gaussian distribution for that component. When the model is fit to data, it estimates soft component probabilities rather than only hard cluster labels.

8.4.1 Soft Cluster Assignments

A major difference between K-means and Gaussian mixture models is that Gaussian mixtures produce soft assignments. Instead of assigning observation $\mathbf{x}_i$ only to the nearest component, the model estimates the probability that $\mathbf{x}_i$ belongs to each component. This probability is called the responsibility of component j for observation i:

$$\gamma_{ij} = P(z_i = j \mid \mathbf{x}_i) = \frac{\pi_j \mathcal{N}\left(\mathbf{x}_i \mid \mu_j, \Sigma_j\right)}{\sum_{\ell=1}^k \pi_\ell \mathcal{N}(\mathbf{x}_i \mid \mu_\ell, \Sigma_\ell)}. \tag{8.28}$$

Here z_i is a latent variable indicating which component generated observation i. The responsibilities satisfy

$$0 \le \gamma_{ij} \le 1, \qquad \sum_{j=1}^k \gamma_{ij} = 1. \tag{8.29}$$

A hard cluster label can still be produced by assigning each observation to the component with the largest responsibility:

$$\hat{z}_i = \arg\max_j \gamma_{ij}. \tag{8.30}$$

Figure 8.23 shows a Gaussian mixture model fit to the wine data using standardized color intensity and proline as the two features. The contours show the fitted mixture density, the dashed curves show the decision regions between mixture components, and the colored points show the component assignments.

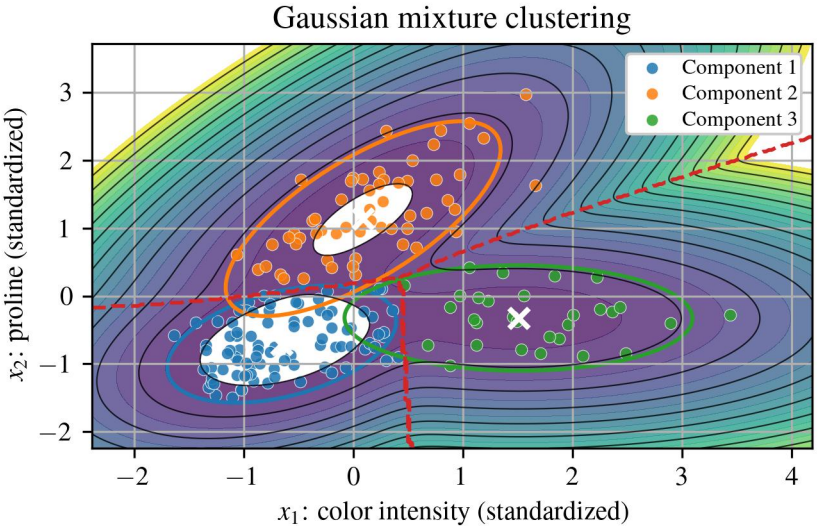

Figure 8.23: Gaussian mixture clustering applied to the wine data using standardized color intensity and proline. Each observation is assigned to the component with the largest posterior probability.

Soft assignments are useful when clusters overlap. An observation near the boundary between two groups may receive probabilities such as 0.55 and 0.45, indicating uncertainty. This is often more informative than a hard label alone, especially when the data contain partially overlapping groups.

8.4.2 Expectation-Maximization

The parameters of a Gaussian mixture model are usually estimated with the expectation-maximization algorithm, abbreviated EM. The objective is to maximize the log-likelihood of the observed data:

$$\ell(\Theta) = \sum_{i=1}^{n} \log \left[\sum_{j=1}^{k} \pi_j \mathcal{N}\left(\mathbf{x}_i \mid \mu_j, \Sigma_j\right) \right], \tag{8.31}$$

where

$$\Theta = \left\{ \pi_j, \mu_j, \Sigma_j \right\}_{j=1}^{k} \tag{8.32}$$

denotes all mixture parameters. Directly maximizing this expression is difficult because the component identity z_i is unknown. EM handles this by alternating between estimating the hidden component memberships and updating the model parameters.

In the expectation step, the responsibilities γ_{ij} are computed using Equation 8.28. In the maximization step, the parameters are updated using weighted averages. Let

$$N_j = \sum_{i=1}^{n} \gamma_{ij} \tag{8.33}$$

be the effective number of observations assigned to component j. The updates are

$$\pi_j = \frac{N_j}{n}, \tag{8.34}$$

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^{n} \gamma_{ij} \mathbf{x}_i, \tag{8.35}$$

and

$$\Sigma_j = \frac{1}{N_j} \sum_{i=1}^{n} \gamma_{ij} \left(\mathbf{x}_i - \mu_j \right) \left(\mathbf{x}_i - \mu_j \right)^T. \tag{8.36}$$

These two steps are repeated until the log-likelihood changes very little between iterations.

8.4.3 Covariance Structure

The covariance matrix controls the shape and orientation of each Gaussian component. This gives Gaussian mixtures more flexibility than K-means. A spherical covariance produces circular clusters in two dimensions. A diagonal covariance allows each feature direction to have a different variance but does not rotate the cluster. A tied covariance uses the same covariance matrix for every component. A full covariance matrix allows each component to have its own tilted ellipse and can capture correlation between features.

Figure 8.24 compares two covariance assumptions on the wine data. With a tied covariance, all components share the same covariance matrix, so the ellipses have the same shape and orientation. With a spherical covariance, each component is forced to be circular in the standardized feature space. These constraints can make the model easier to estimate, but they may also prevent the model from matching the shape of the data.

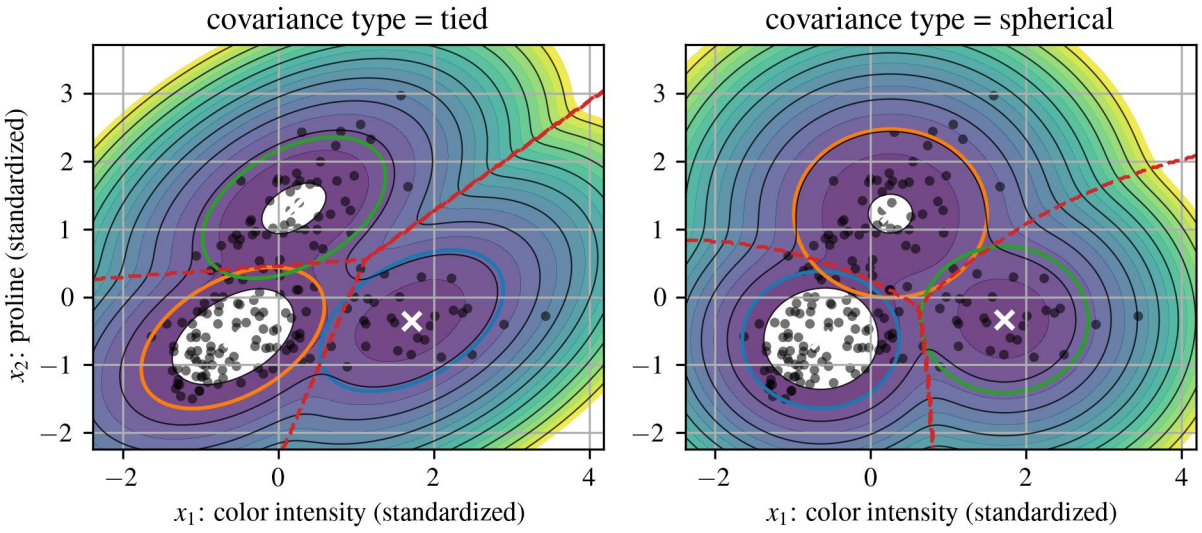

Figure 8.24: Effect of covariance structure on Gaussian mixture components fit to the wine data. The tied model shares one covariance matrix across all components, while the spherical model restricts each component to have circular contours in the standardized feature space.

The covariance choice is important because it controls model flexibility. A full covariance model can fit complex elliptical clusters, but it has more parameters and can overfit small data sets. A spherical or diagonal covariance model is more restrictive, but it may be more stable when the number of observations is limited.

Gaussian mixture models also require choosing the number of components, k. The elbow rule can be used, but likelihood-based criteria are common because GMMs are probabilistic models. Two common choices are the Akaike information criterion (AIC) and Bayesian information criterion (BIC):

$$\text{AIC} = 2p - 2\ell(\hat{\Theta}), \tag{8.37}$$

$$\text{BIC} = p\log(n) - 2\ell(\hat{\Theta}), \tag{8.38}$$

where p is the number of estimated parameters and $\ell(\hat{\Theta})$ is the maximized log-likelihood. Lower values indicate a better tradeoff between model fit and model complexity. BIC penalizes model complexity more strongly than AIC when n is large.

8.4.4 Relationship Between K-Means and Gaussian Mixtures

K-means can be viewed as a limiting case of a Gaussian mixture model. If all mixture components have spherical covariance matrices with the same small variance, then assigning a point to the component with the largest probability becomes similar to assigning it to the nearest mean. In that sense, K-means is like a hard-assignment, equal-variance version of a Gaussian mixture model.

This connection is useful because it explains the behavior of both methods. K-means is simpler and often faster, but it cannot represent uncertainty in cluster membership. Gaussian mixtures are more flexible and probabilistic, but they require estimating more parameters and can be sensitive to initialization, covariance assumptions, and the number of components.

8.5 Anomaly Detection

Clustering methods are often used not only to find groups, but also to identify observations that do not fit any group well. These unusual observations are called anomalies, outliers, or novelties depending on the application. In engineering, anomalies may correspond to faulty sensors, rare operating conditions, damaged components, unusual loading events, or data-entry errors.

A simple anomaly detection idea is to fit a model to the normal data and then flag observations that have low likelihood under that model. For a single Gaussian distribution, the density of an observation is

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \mu, \Sigma). \tag{8.39}$$

An anomaly can be defined by applying a threshold τ:

$$\hat{y}(\mathbf{x}) = \begin{cases} \text{anomaly}, & p(\mathbf{x}) < \tau, \\ \text{normal}, & p(\mathbf{x}) \geq \tau. \end{cases} \tag{8.40}$$

Equivalently, since log-likelihoods are easier to compute numerically, the threshold can be applied to the log-likelihood:

$$\log p(\mathbf{x}) < \log \tau. \tag{8.41}$$

For a Gaussian model, anomaly detection is closely related to the Mahalanobis distance,

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x}-\mu)^T \Sigma^{-1} (\mathbf{x}-\mu)}. \tag{8.42}$$

This distance measures how far $\mathbf{x}$ is from the mean after accounting for the covariance of the data. A point may look far away in Euclidean distance but not be unusual if it lies along a high-variance direction. Conversely, a point may have moderate Euclidean distance but be highly unusual if it lies along a low-variance direction.

Gaussian mixture models extend this idea to data with multiple normal operating regimes. Instead of using one Gaussian density, the mixture density from Equation 8.25 is used. An observation is considered normal if it has high likelihood under at least one component and anomalous if it has low likelihood under the entire mixture. This is useful when a system has several legitimate operating states. A point may be far from one operating state but close to another, so a single Gaussian model may be too restrictive.

Figure 8.25 shows this idea using the wine data. The Gaussian mixture model is fit to the standardized color intensity and proline features. Observations with low likelihood under the fitted mixture are marked as potential anomalies. These points are not necessarily "bad" measurements; they are simply observations that the fitted model considers unusual relative to the rest of the data.

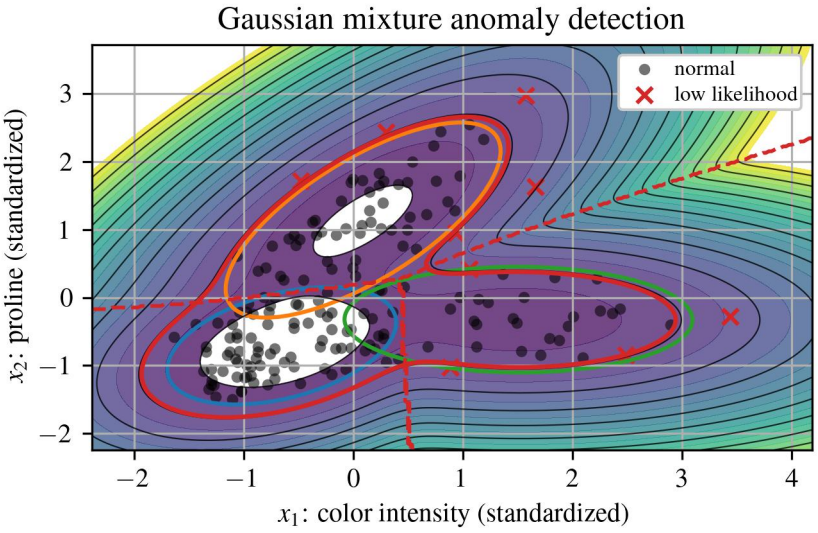

Figure 8.25: Gaussian mixture anomaly detection applied to the wine data. Observations in low-likelihood regions of the fitted mixture model are flagged as potential anomalies.

The threshold τ controls the tradeoff between false alarms and missed anomalies. A high threshold labels more observations as anomalous, increasing sensitivity but also increasing false positives. A low threshold labels fewer observations as anomalous, reducing false positives but potentially missing subtle faults. In practice, τ may be selected using a validation set, engineering limits, or an assumed contamination rate representing the expected fraction of anomalies.

In scikit-learn, `sklearn.covariance.EllipticEnvelope` can be used for Gaussian anomaly detection can when the normal data are reasonably Gaussian. More general anomaly detection methods include `IsolationForest`, `LocalOutlierFactor`, and `OneClassSVM`. These methods use different assumptions, but the modeling goal is the same: learn what normal data look like and identify observations that deviate from that structure.

Clustering and anomaly detection are therefore closely related. Clustering asks, "Which observations belong together?" Anomaly detection asks, "Which observations do not belong with the rest?" Both questions are unsupervised, both depend strongly on the feature representation, and both require careful interpretation. The output of the algorithm should be treated as evidence of structure in the data, not as a final physical explanation. The final step is always to connect the discovered clusters or anomalies back to the engineering system that produced the measurements.